Pattern Recognition and Machine Learning

Chapter 1: Introduction

Reading Notes

October 26, 2026

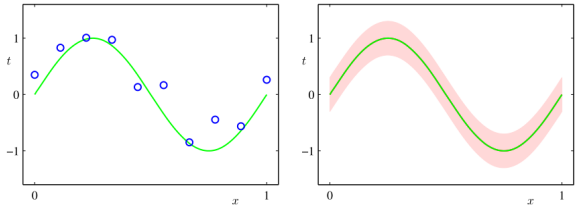
Table of Contents

1. Example: Polynomial Curve Fitting
2. Probability Theory
3. Model Selection
4. Curve of Dimensionality
5. Decision Theory
6. Information Theory

Polynomial Curve Fitting Problem

Goal: Predict target t from input x

Data: $t = \sin(2\pi x) + \varepsilon$, where $\varepsilon \sim \mathcal{N}(0, \sigma^2)$



Model: M-th order polynomial

$$y(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \ldots + w_M x^M = \sum_{j=0}^{M} w_j x^j$$

Error Function and Solution

Sum-of-squares Error Function:

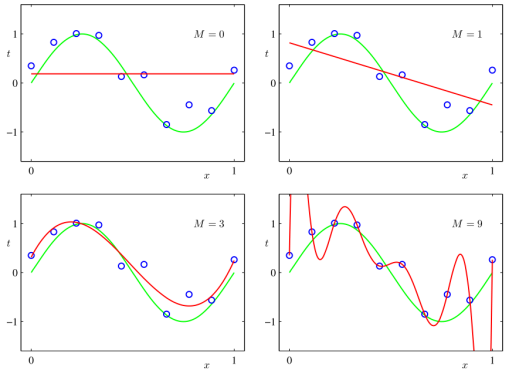
$$E(\boldsymbol{w}) = \frac{1}{2} \sum_{n=1}^{N} \{y(x_n, \boldsymbol{w}) - t_n\}^2$$

Solution: Minimize the Error Function

$$\boldsymbol{w}^* = \arg\min_{\boldsymbol{w}} E(\boldsymbol{w})$$

- Standard choice for regression problems
- Intuitive measure of prediction accuracy
- Provides optimal parameter estimates

Model Performance Analysis



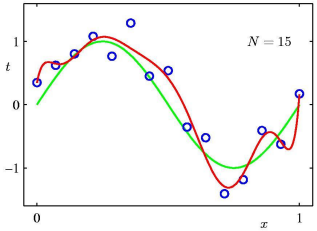
Polynomial fits of different orders

Bias-Variance Trade-off

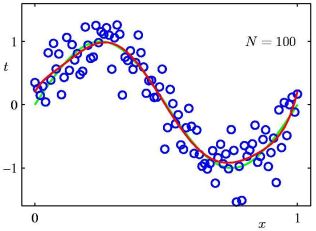
- **Under-fitting (M=0,1)**:
  - High bias: model too simple
  - Cannot capture true pattern
  - Poor performance on both train and test
- **Good fit (M=3)**:
  - Balanced bias and variance
  - Captures true underlying function
  - Good generalization
- **Over-fitting (M=9)**:
  - High variance: model too complex
  - Fits noise in training data
  - Poor generalization to test data

Addressing Over-fitting: More Data

Solution: Increase training data size N



$N = 15$ (Small dataset)



$N = 100$ (Large dataset)

- **Small** N: High variance, over-fitting likely
- **Large** N: Better generalization, reduced over-fitting
- More data helps model learn true underlying pattern

Key Insight

Increasing training data is often the most effective (but expensive) solution to over-fitting

Addressing Over-fitting: Regularization

Regularized Error Function:

$$\tilde{E}(\boldsymbol{w}) = \frac{1}{2}\sum_{n=1}^{N}\{y(x_n, \boldsymbol{w}) - t_n\}^2 + \frac{\lambda}{2}\|\boldsymbol{w}\|^2$$

- λ controls regularization strength
- Penalizes large coefficients
- Encourages simpler models
- Helps prevent over-fitting

Trade-off: Need to choose appropriate λ value

Limiting Cases of λ

$\lambda \to 0$: no regularization $\Rightarrow$ over-fitting
$\lambda \to \infty$: fit collapses to a constant $\Rightarrow$ under-fitting

Bayesian vs Frequentist Probability

Frequentist View

- Probabilities are frequencies of random, repeatable events
- Parameters are fixed but unknown
- No incorporation of prior knowledge
- Focus on maximum likelihood estimation

Bayesian View

- Probabilities quantify uncertainty
- Parameters are random variables with distributions
- Incorporates prior knowledge through prior distributions
- Focus on posterior distributions

Bayesian Learning Framework

Bayes' Theorem:

$$\text{posterior} \propto \text{likelihood} \times \text{prior}$$

$$p(\boldsymbol{w}|\mathcal{D}) = \frac{p(\mathcal{D}|\boldsymbol{w})p(\boldsymbol{w})}{p(\mathcal{D})}$$

- **Prior** $p(\boldsymbol{w})$: Initial belief about parameters
- **Likelihood** $p(\mathcal{D}|\boldsymbol{w})$: How well parameters explain data
- **Posterior** $p(\boldsymbol{w}|\mathcal{D})$: Updated belief after seeing data
- **Evidence** $p(\mathcal{D})$: Normalization constant $\quad p(\mathcal{D}) = \int p(\mathcal{D}|\boldsymbol{w})p(\boldsymbol{w})d\boldsymbol{w}$

Advantage: Naturally incorporates uncertainty and prior knowledge

Gaussian Distribution and Maximum Likelihood

Likelihood for i.i.d. data:

$$p(\mathcal{D}|\mu, \sigma^2) = \prod_{n=1}^{N} \mathcal{N}(x_n|\mu, \sigma^2)$$

Log-Likelihood:

$$\ln p(\mathcal{D}|\mu, \sigma^2) = -\frac{N}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{n=1}^{N} (x_n - \mu)^2$$

MLE Estimates:

$$\mu_{\mathrm{ML}} = \frac{1}{N} \sum_{n=1}^{N} x_n$$

$$\sigma^2_{\mathrm{ML}} = \frac{1}{N} \sum_{n=1}^{N} (x_n - \mu_{\mathrm{ML}})^2$$

Variance estimate is biased! Use $N - 1$ for unbiased estimate

Probabilistic Curve Fitting

Model with Gaussian Noise:

$$t = y(x, \boldsymbol{w}) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \beta^{-1})$$

Likelihood Function:

$$p(\boldsymbol{t}|\boldsymbol{X}, \boldsymbol{w}, \beta) = \prod_{n=1}^{N} \mathcal{N}(t_n|y(x_n, \boldsymbol{w}), \beta^{-1})$$

Log-Likelihood:

$$\ln p(\boldsymbol{t}|\boldsymbol{X}, \boldsymbol{w}, \beta) = \frac{N}{2} \ln \beta - \frac{N}{2} \ln(2\pi) - \frac{\beta}{2} \sum_{n=1}^{N} (t_n - y(x_n, \boldsymbol{w}))^2$$

Key Insight

Maximizing likelihood = Minimizing sum-of-squares error

Bayesian Approach and Regularization

Gaussian Prior:

$$p(\boldsymbol{w}|\alpha) = \mathcal{N}(\boldsymbol{w}|\boldsymbol{0}, \alpha^{-1}\boldsymbol{I})$$

Posterior Distribution:

$$p(\boldsymbol{w}|\boldsymbol{X}, \boldsymbol{t}, \alpha, \beta) \propto p(\boldsymbol{t}|\boldsymbol{X}, \boldsymbol{w}, \beta)p(\boldsymbol{w}|\alpha)$$

Log-Posterior:

$$\ln p(\boldsymbol{w}|\boldsymbol{X}, \boldsymbol{t}, \alpha, \beta) = -\frac{\beta}{2}\sum_{n=1}^{N}(t_n - y(x_n, \boldsymbol{w}))^2 - \frac{\alpha}{2}\boldsymbol{w}^T\boldsymbol{w} + \text{const}$$

MAP Estimate:

$$\boldsymbol{w}_{\text{MAP}} = \arg\min_{\boldsymbol{w}} \left[\frac{\beta}{2}\sum_{n=1}^{N}(t_n - y(x_n, \boldsymbol{w}))^2 + \frac{\alpha}{2}\boldsymbol{w}^T\boldsymbol{w}\right]$$

Fundamental Insight

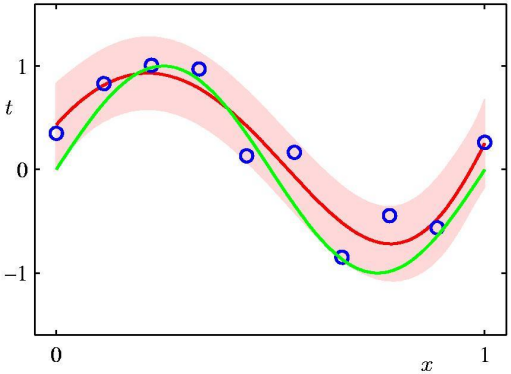
Regularization emerges naturally from Bayesian priors!

Bayesian Curve Fitting: Predictive Distribution

Predictive Distribution:

$$p(t|x, X, T) = \int p(t|x, \boldsymbol{w}) p(\boldsymbol{w}|X, T) d\boldsymbol{w}$$

- **Model averaging**: Over all possible parameters
- **Uncertainty**: Naturally incorporated
- **Complexity**: Automatic control prevents over-fitting
- **Distribution**: Full predictive distribution provided



Bayesian predictive mean

Predictive uncertainty

Key Advantage

Robust uncertainty quantification through principled probabilistic framework

Model Selection Techniques

Cross-Validation (Brief)

- Split data into training and validation sets
- Rotate splits to get robust performance estimate
- Prevents over-fitting to specific data split

Information Criteria

- **AIC**: $\ln p(\mathcal{D}|\boldsymbol{w}_{\mathrm{ML}}) - M$
- **BIC**: $\ln p(\mathcal{D}|\boldsymbol{w}_{\mathrm{ML}}) - \frac{1}{2} M \ln N$
- Balance model fit and complexity
- No need for separate validation set

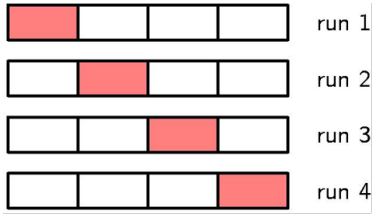


Figure: Cross-Validation Process

Key Idea

Penalize model complexity to improve generalization

High-Dimensional Polynomial Complexity

D-dimensional 3rd-order Polynomial:

$$y(\boldsymbol{x}, \boldsymbol{w}) = w_0 + \sum_{i=1}^{D} w_i x_i + \sum_{i=1}^{D} \sum_{j=i}^{D} w_{ij} x_i x_j + \sum_{i=1}^{D} \sum_{j=i}^{D} \sum_{k=j}^{D} w_{ijk} x_i x_j x_k$$

Parameter Count Analysis:

$$\begin{aligned} \text{Total parameters} &= 1 + D + \frac{D(D+1)}{2} + \frac{D(D+1)(D+2)}{6} \\ &= \frac{(D+1)(D+2)(D+3)}{6} = \mathcal{O}(D^3) \end{aligned}$$

The Curse Manifested

Cubic parameter growth $\Rightarrow$ Exponential data requirements

Curse of Dimensionality

Fundamental Challenge: Difficulties in high-dimensional spaces

Low Dimensions (2D/3D)

- Intuitive geometric interpretation
- Data points are relatively dense
- Distance metrics work well
- Simple models often sufficient

High Dimensions (100D+)

- Counter-intuitive geometry
- Data becomes extremely sparse
- Distance measures lose meaning
- Complex models required

Exponential Sampling Problem

To sample a d-dimensional unit cube with resolution ε in each dimension:

Required samples: $N = \left(\frac{1}{\varepsilon}\right)^d$

Example: $\varepsilon = 0.1$ in 10D $\Rightarrow N = 10^{10}$ samples needed!

Geometry in High Dimensions

Our 3D intuition fails — three counter-intuitive facts in $\mathbb{R}^D$:

- **Volume concentrates in a thin shell** (sphere volume $V_D(r) = K_D r^D$)

$$\frac{V_D(1) - V_D(1-\varepsilon)}{V_D(1)} = 1 - (1-\varepsilon)^D \xrightarrow{D\to\infty} 1$$

- **Volume concentrates near the corners** (Stirling: $\Gamma(x+1) \approx \sqrt{2\pi x}\,(x/e)^x$)

$$\frac{\text{volume of sphere}}{\text{volume of cube}} = \frac{\pi^{D/2}}{2^D\,\Gamma(D/2+1)} \xrightarrow{D\to\infty} 0$$

- **Random directions become nearly orthogonal**

$$|\cos\theta| \sim \frac{1}{\sqrt{D}} \xrightarrow{D\to\infty} 0$$

Consequence: distances lose contrast

The formulas above imply $(d_{\max} - d_{\min})/d_{\min} \to 0 \Rightarrow$ "nearest" and "farthest" become indistinguishable

Coping with High Dimensions

Traditional Approaches

- Dimensionality Reduction
  - PCA, manifold learning
  - Explicit feature compression
- Feature Selection
  - Statistical filtering
  - Sparse models (L1)
- Regularization
  - L2 regularization
  - Early stopping

Deep Learning Approaches

- Autoencoders
  - Learn compressed representations
  - Unsupervised dimensionality reduction
- Hierarchical Feature Learning
  - Automatic feature abstraction
  - Progressive complexity reduction
- Built-in Regularization
  - Dropout techniques
  - Weight sharing

Deep Learning Advantage

End-to-end feature learning: Automatically discovers task-relevant representations without manual feature engineering

From Inference to Decision

Decision regions $\mathcal{R}_j$: assign every $\mathbf{x} \in \mathcal{R}_j$ to class j; L_{kj} is the cost of assigning an input of class k to class j, and errors need not cost the same

Expected loss of a partition $\{\mathcal{R}_j\}$:

$$\mathbb{E}[L] = \sum_k \sum_j \int_{\mathcal{R}_j} L_{kj}\, p(\mathbf{x}, \mathcal{C}_k)\, \mathrm{d}\mathbf{x}$$

The only choice is the partition. Because $p(\mathbf{x}) \geq 0$, we can minimize the expected loss point by point: each $\mathbf{x}$ should be assigned to its least-loss class,

$$\arg\min_j \sum_k L_{kj}\, p(\mathcal{C}_k|\mathbf{x})$$

Reject Option

Optimal is not the same as confident: if $\max_j p(\mathcal{C}_j|\mathbf{x}) < \theta$, abstain rather than guess

Decision-making often looks beyond the probabilities; that is what separates it from inference.

Comparison of Decision Approaches

Minimizing Misclassification	**Minimizing Expected Loss**
$\arg\min_j \sum_{k \neq j} p(\mathcal{C}_k \mid \mathbf{x})$	$\arg\min_j \sum_k L_{kj} p(\mathcal{C}_k \mid \mathbf{x})$
Equal error costs	Asymmetric error costs
Special case: $L_{kj} = 1 - \delta_{kj}$	General loss matrix
Simple implementation	Flexible cost structure

Mathematical Equivalence

$\arg\max_j p(\mathcal{C}_j \mid \mathbf{x}) = \arg\min_j \sum_{k \neq j} p(\mathcal{C}_k \mid \mathbf{x})$
Both minimize misclassification rate under 0-1 loss

Loss Matrix Example:

	Predict Healthy	**Predict Sick**
Actual Healthy	0	10
Actual Sick	100	0

Generative vs Discriminative Models

Generative Models

- Model joint probability $p(\boldsymbol{x}, \mathcal{C}_k)$
- Learn data generation process: $p(\boldsymbol{x}, \mathcal{C}_k) = p(\boldsymbol{x}|\mathcal{C}_k)p(\mathcal{C}_k)$
- Can synthesize new data samples
- Handle missing data naturally
- Examples: Naive Bayes, Gaussian Mixture Models
- Advantage: Provide full probabilistic framework, adaptable to outlier or novelty detection

Discriminative Models

- Model conditional probability $p(\mathcal{C}_k|\boldsymbol{x})$ directly
- Learn decision boundaries between classes
- Often more efficient for pure classification
- Examples: Logistic Regression, Neural Networks
- Advantage: Optimize the target task directly and efficiently

Note: a modern unconditional generative model gives only $p(\boldsymbol{x})$, not $p(\mathcal{C}_k|\boldsymbol{x})$, so it cannot serve directly as a classifier.

Quantifying Uncertainty

Information Content

- Measures information required to resolve uncertainty
- For event x: $h(x) = -\log_2 p(x)$ (bits)
- Rare events (low probability) have high information content
- Common events provide little information

Entropy - Average Uncertainty

- Discrete: $H[x] = -\sum_x p(x) \log_2 p(x)$
- Continuous: $H[x] = -\int p(x) \ln p(x) dx$
- Maximum entropy = maximum uncertainty
- Measures average surprise

Relative entropy and mutual information

KL Divergence: Measures distribution difference

$$\mathrm{KL}(p\|q) = \int p(x) \ln \frac{p(x)}{q(x)} dx$$

- **Application**: Model comparison, variational inference
- **Property**: $\mathrm{KL}(p\|q) \geq 0$, asymmetric

Mutual Information: Measures variable dependence

$$I[\boldsymbol{x}, \boldsymbol{y}] = \iint p(\boldsymbol{x}, \boldsymbol{y}) \ln \left(\frac{p(\boldsymbol{x}, \boldsymbol{y})}{p(\boldsymbol{x}) p(\boldsymbol{y})} \right) d\boldsymbol{x} d\boldsymbol{y}$$

$$I[\boldsymbol{x}, \boldsymbol{y}] = H[\boldsymbol{x}] + H[\boldsymbol{y}] - H[\boldsymbol{x}, \boldsymbol{y}]$$

- **Application**: Feature selection, dependency analysis
- **Property**: Symmetric, $I[\boldsymbol{x}, \boldsymbol{y}] \geq 0$

Summary

- **Curve Fitting**: Illustrates over-fitting, model selection, and regularization
- **Bayesian Methods**: Provide natural framework for regularization and uncertainty
- **Model Selection**: Crucial for generalization performance
- **Curse of Dimensionality**: Fundamental challenge in high-dimensional spaces
- **Decision Theory**: Formal foundation for classification tasks
- **Information Theory**: Mathematical framework for uncertainty quantification

Key Takeaway

These foundational concepts provide the basis for understanding more advanced machine learning techniques and algorithms.